

D. Wordbot Prototype Documentation

Hardware Setup

- Raspberry Pi: A Raspberry Pi (model specified by your setup) serves as the core processing unit. It runs the Python scripts controlling sensors, audio, and language model interaction.
- Proximity Sensor: The VCNL4040 sensor is connected via I2C bus to detect user presence without physical contact. The sensor triggers state changes by measuring proximities above a configurable threshold.
- Display: The Mini PiTFT ST7789 display is used for visual feedback, showing system states through background colors (green for idle, red for listening, yellow for processing) and textual status with proximity readings.
- Audio Input/Output: A USB or Pi-compatible microphone captures speech input. Audio output is managed via either small USB speakers or the Pi's audio jack, used for text-to-speech playback through gTTS-generated audio.
- Other hardware peripherals: GPIO pins are used to control the display backlight and may interface with additional buttons or LEDs for system feedback.

Software Architecture

- Programming Language: The system is implemented in Python, leveraging various libraries to manage hardware and AI components.
- Speech Processing: Speech-to-text functionality uses the SpeechRecognition package with Google API backend for accurate transcription. A dedicated microphone device index is configured for capture.
- Text-to-Speech: gTTS is employed to synthesize speech from text prompts, converting dialogue content into audio for user playback.
- Language Model Integration: Ollama client library communicates with a local LLM instance, generating structured responses including the Word of the Day, its definition, example usage, and optional etymology on demand.
- Sensor Management: Proximity detection triggers transitions from sleep to active states, guiding the listening and dialogue stages.
- Display Control: Using the Adafruit ST7789 Python library, color-coded statuses are rendered on the Mini PiTFT display to visually represent system states and sensor values.
- State Machine and Control Logic: A custom state machine manages the interaction flow: Idle (waiting for proximity), Listening (awaiting "yes"), Word Delivery, Etymology Offer, and Reset. Transitions are driven by sensor inputs, speech detection, and user responses.
- Error and Exception Handling: The system includes safeguards against speech recognition timeouts, LLM connectivity failures, and sensor errors, reverting to safe idle or retry states when issues occur.

Integration and Communication

- I2C Bus: Used for communication between the Raspberry Pi and VCNL4040 proximity sensor.
- SPI Bus: Used for driving the Mini PiTFT ST7789 display with specified chip select and data/command pins.

- Python Software Modules: Orchestrate hardware interaction and AI dialogue generation, updating display and audio output based on real-time sensor and speech data.
- Data Flow: Sensor data triggers state changes; speech input is transcribed and sent as prompts to Ollama; responses are parsed and converted to speech; system status is reflected on the display.

Deployment and Setup Instructions

- Flash Raspberry Pi OS Lite (64-bit recommended) onto an SD card.
- Enable I2C and SPI interfaces using `raspi-config`.
- Connect VCNL4040 sensor to I2C pins and Mini PiTFT to SPI pins.
- Attach microphone and speakers, configuring audio devices (`arecord -l`, `aplay -l`, and `alsamixer`).
- Install Python dependencies: Adafruit libraries, SpeechRecognition, gTTS, Pillow, Ollama client.
- Configure microphone device index and Ollama host in the main script.
- Run the WordBot Python script; monitor display and audio outputs.

Testing and Validation

- Initial testing with sensor and RGB test script confirmed proximity-triggered visual feedback.
- Speech recognition and TTS were tested using direct audio prompts before full integration.
- Wizard of Oz testing informed redesigns to manage latency and user uncertainty.

Future Improvements

- Adding physical buttons for reset and manual control.
- Incorporating additional sensors like ambient light and microphone noise measurement.
- Implementing pre-loading strategies to mitigate LLM latency.
- Developing a web-based controller for improved wizarding and debugging.